

Program-4

Aim

To load the **Iris Dataset** and implement the **k-Nearest Neighbors** algorithm for classifying flowers based on their features.

To split the dataset into training and testing sets and evaluate the model using **accuracy and F1-score**.

To test the algorithm with different values of **k (1, 3, 5)** and compare the performance of **regular k-NN and weighted k-NN**.

Theory

Machine Learning is a branch of artificial intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed. One of the most common machine learning tasks is **classification**, where the goal is to assign data into predefined categories.

The Iris dataset is one of the most widely used datasets for learning classification techniques. It consists of **150 samples of iris flowers** belonging to three different species: **Setosa, Versicolor, and Virginica**. Each flower sample has four numerical features: sepal length, sepal width, petal length, and petal width. These features are used as input variables to classify the flower species.

The **k-Nearest Neighbors (k-NN)** algorithm is a supervised learning algorithm used for classification and regression problems. It works based on the concept that similar data points are located close to each other in the feature space. When a new data point needs to be classified, the algorithm calculates the distance between the new data point and all the training data points. It then selects the **k nearest neighbors** and assigns the class that occurs most frequently among those neighbors.

The similarity between data points is usually measured using **Euclidean distance**, which calculates the straight-line distance between two points in multidimensional space.

In machine learning experiments, the dataset is typically divided into **training and testing sets**. The training set is used to train the model, while the testing set is used to evaluate how well the model performs on unseen data.

To measure the performance of the model, evaluation metrics such as **accuracy and F1-score** are used. Accuracy measures the percentage of correctly classified instances, while the F1-score provides a balanced measure of precision and recall.

The value of **k** plays an important role in the k-NN algorithm. A very small value of k (such as 1) may lead to overfitting and sensitivity to noise, whereas a moderate value (such as 3 or 5) usually produces more reliable results.

An extension of the k-NN algorithm is **weighted k-NN**, where the influence of each neighbor depends on its distance from the query point. In this approach, closer neighbors are given more importance than distant ones. A commonly used weighting formula is:

$$\text{Weight} = 1 / d^2$$

where **d** represents the distance between the data points. This method improves classification accuracy by emphasizing nearby data points.

Thus, the k-NN algorithm is a simple, effective, and widely used method for classification problems.

Procedure

1. Import the necessary Python libraries such as NumPy, pandas, and scikit-learn.
2. Load the Iris dataset from the sklearn dataset library.
3. Convert the dataset into a DataFrame and separate the **features (X)** and **target variable (y)**.
4. Split the dataset into **training data and testing data** using train-test split.
5. Apply the k-NN algorithm with different values of **k (1, 3, 5)**.
6. Train the model using the training dataset.
7. Predict the species of flowers using the testing dataset.
8. Evaluate the model using **accuracy score and F1-score**.
9. Implement **weighted k-NN** using distance-based weights.
10. Compare the performance of regular k-NN and weighted k-NN.

Result

The **Iris Dataset** was successfully loaded and the **k-Nearest Neighbors** algorithm was implemented for flower classification. The dataset was divided into training and testing sets and evaluated using **accuracy and F1-score**.

The model was tested for different values of **k (1, 3, 5)**. It was observed that moderate values of k produced more stable results. The **weighted k-NN** method also showed comparable or slightly improved performance compared to regular k-NN.